

Project ODIN

UI & Backend Technical Breakdown

Author: Tan Yi Rong | GlobalFoundries – AMHS Department

 *How to use this document: Each section shows a plain-English description on the LEFT and the exact code on the RIGHT. The file path and line numbers are shown above each table so you can jump straight to it in VS Code (Ctrl+G → type line number).*

1. System Overview

The Project ODIN dashboard is made up of two main files that work together: the Backend (app.py) which is the "brain" that talks to the physical sensor and runs a mini web server, and the Frontend (index.html) which is the "face" — the dashboard you see in the browser.

Part	File
Backend (Brain)	OHT Wheel Measurement Project/app.py
Frontend (Face)	OHT Wheel Measurement Project/templates/index.html

2. Backend — app.py

2.1 Importing the Tools

 **app.py** → Lines 1 – 7

Before the code can do anything, it loads its required tools (called "libraries"). Think of these like apps on your phone — you open them before using them.

- socket — The "telephone line" to the physical OMRON sensor.
- threading — Runs two tasks at the same time.
- flask — Hosts the dashboard as a local website.
- csv — Saves data to a spreadsheet file.
- datetime — Stamps each reading with the current time.

```
import socket
import time
import csv
import threading
from datetime import datetime
from collections import deque
from flask import Flask, render_template, jsonify
```

2.2 Configuration Settings

 **app.py** → Lines 21 – 26

All the key project settings are stored in one place at the top of the file, making them easy to update without hunting through the code.

- SENSOR_IP / SENSOR_PORT — Network address of the OMRON ZP-EIP sensor.
- CSV_FILENAME — The spreadsheet file where all data is saved permanently.
- WHEEL_BASE_DIAMETER_MM — A healthy new wheel is exactly 125mm. The sensor is calibrated to this point.
- WHEEL_DETECT_THRESHOLD_MM — If a reading is below 40mm, the code knows a wheel is passing.

```
SENSOR_IP = '192.168.250.1'
SENSOR_PORT = 64000
CSV_FILENAME =
'OHT_Wheel_Predictive_Maintenance_Log.csv'

WHEEL_BASE_DIAMETER_MM = 125.0
WHEEL_DETECT_THRESHOLD_MM = 40.0
```

2.3 Live Memory Variables

 **app.py** → Lines 28 – 33

These are the shared "memory" variables that hold live data.

```
current_distance = 0.0

# Keep the last 100 readings for the live
chart
```

- `current_distance` — Always holds the very latest sensor reading in mm.
- `distance_history` — A rolling list of the last 100 readings, like a snake: new readings come in at the head and old ones fall off the tail. This is what feeds the scrolling live graph on the dashboard.

```
distance_history = deque([0.0]*100,
maxlen=100)
```

2.4 Talking to the Sensor

 **app.py** → Lines 39 – 61

This function sends a command to the OMRON sensor and decodes the reply.

1. It sends the text "MS,01,0" — asking for the latest measurement.
2. The sensor replies in hexadecimal (e.g. FFFF9BCD).
3. The code converts hex → integer → millimetres.
4. If the sensor has nothing in range, it returns 999.0 as a placeholder.

The key formula:

`int_val / 100000.0`

...converts the raw integer to millimetres (sensor outputs in 0.01-micrometre steps).

```
def send_omron_command(sock):
    command_str = "MS,01,0\r\n"

    sock.sendall(command_str.encode('ascii'))
    response =
sock.recv(1024).decode('ascii').strip()

    parts = response.split(',')
    if len(parts) >= 3 and parts[0] ==
'MS':
        hex_value_str = parts[2].strip()
        int_val = int(hex_value_str, 16)

        # Out of range check
        if 0x7FFF0000 <= int_val <=
0x7FFFFFFF:
            return 999.0
        if int_val >= 0x80000000:
            int_val -= 0x100000000

        mm_value = int_val / 100000.0
        return mm_value
    return 999.0
```

2.5 Main Sensor Loop — Detecting Wheels

 **app.py** → Lines 63 – 123

This is the heart of the system. It runs forever in the background.

- Lines 75–79: Opens a network connection to the sensor. If it fails, waits 2 seconds and retries.
- Line 86: Calls `send_omron_command()` 100 times per second (every 10ms).
- Lines 90–95: If reading is between -40mm and +40mm → wheel is in view! Track the peak.
- Lines 97–117: When the reading jumps back above 40mm → wheel has passed!
 - Line 104: `wheel_diameter = 125mm + peak_reading` (the core formula).
 - Lines 108–111: Simulates 4 wheels by adding ±0.02mm random noise to the 1 real reading.
 - Lines 113–115: Saves one new row to the CSV file.
- Line 119: `sleep(0.01)` — polls 100 times per second.

```
# Simplified view of the detection logic:
while True:
    dist = send_omron_command(sock)
    current_distance = dist
    distance_history.append(dist)

    if -40.0 < current_distance < 40.0:
        # Wheel is in view — track peak
        if not wheel_in_view:
            wheel_in_view = True
            max_dist = current_distance
        if current_distance > max_dist:
            max_dist = current_distance

    elif wheel_in_view and
abs(current_distance) >= 40.0:
        # Wheel has passed — calculate &
save
        wheel_diameter =
WHEEL_BASE_DIAMETER_MM + max_dist
        fl = round(wheel_diameter, 3)
        fr = round(wheel_diameter +
random.uniform(-0.015, 0.015), 3)
        # ... save to CSV
        wheel_in_view = False
```

```
time.sleep(0.01)
```

2.6 Web Server & API Endpoints

 **app.py** → Lines 125 – 155

Flask creates three web "addresses" (called routes) that the browser can call:

- `"/` (Line 125) — Serves the main `index.html` dashboard page.
- `/api/live` (Line 129) — Returns the last 100 live sensor readings as JSON for the scrolling chart.
- `/api/history` (Line 143) — Opens the CSV file and returns up to the last 1,000 wheel records as JSON for the Fleet and Analysis tabs.

```
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/api/live')
def api_live():
    recent = list(distance_history)
    return jsonify({
        'left': { 'current_distance':
current_distance,
                'history': recent },
        'right': { 'current_distance':
current_distance,
                'history': recent }
    })

@app.route('/api/history')
def api_history():
    # Reads CSV and returns last 1000
    records
    ...
```

2.7 Starting Everything Up

 **app.py** → Lines 157 – 163

When `app.py` is run, two workers start simultaneously using Python threading:

1. A background THREAD runs `sensor_polling_thread()` — silently reads sensor data 100x/sec forever.
2. The Flask WEB SERVER starts on port 5000, waiting for the browser to connect.

Think of it like two factory workers starting at the same time: one keeps measuring wheels non-stop, the other stands at the front desk answering browser requests instantly.

```
if __name__ == '__main__':
    # Start sensor background thread
    t = threading.Thread(
        target=sensor_polling_thread,
        daemon=True
    )
    t.start()

    # Start Flask web server
    app.run(host='0.0.0.0', port=5000,
            debug=False)
```

3. Frontend — index.html

3.1 Loading the Charting Library

 [index.html](#) → Lines 8

This single line loads Chart.js — a free, popular library that draws all the live graphs. Without this, all the chart areas on the page would just be blank boxes.

```
<script
src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

3.2 The Dark Theme & Color System

 [index.html](#) → Lines 10 – 18

CSS variables define the entire color palette in one place. Changing a variable here automatically updates every part of the page that uses it.

- `--bg-color: #0b0d17` — The deep navy-black background.
- `--accent-primary: #00f0ff` — The glowing cyan color used for live data.
- `--accent-secondary: #ff0055` — The red/pink used for warnings and thresholds.

```
:root {
  --bg-color: #0b0d17;
  --panel-bg: rgba(20, 24, 44, 0.6);
  --accent-primary: #00f0ff; /* Cyan */
  --accent-secondary: #ff0055; /* Red */
  --text-main: #ffffff;
  --text-muted: #a0a5c0;
}
```

3.3 The Pulsing "SYSTEM ONLINE" Dot

 [index.html](#) → Lines 62 – 75

The glowing dot in the header uses a CSS animation called "pulse".

`@keyframes` defines a mini-animation that makes the dot repeatedly grow (scale 1.2x) and shrink (scale 0.95x) every 1.5 seconds, creating the heartbeat effect. This visually communicates that the system is live and actively running.

```
.live-dot {
  animation: pulse 1.5s infinite;
}

@keyframes pulse {
  0% { transform: scale(0.95); opacity: 0.8; }
  50% { transform: scale(1.2); opacity: 1; }
  100% { transform: scale(0.95); opacity: 0.8; }
}
```

3.4 The Three Tab Buttons

 [index.html](#) → Lines 257 – 261 (HTML) & 375 – 386 (JS)

Each button calls the `switchTab()` JavaScript function (defined at Line 375). That function simply shows one `<div>` panel and hides the others by toggling CSS display properties. There is no page reload — the content swaps instantly.

```
<button onclick="switchTab('live')">Live
Monitoring</button>
<button onclick="switchTab('fleet')">Fleet
Overview</button>
<button onclick="switchTab('oht')">OHT
Analysis</button>
```

```
// switchTab logic (Line 375):
function switchTab(tab) {
  document.getElementById('tab-
live').style.display
    = tab === 'live' ? 'block' :
```

```
'none';
// ... same for fleet and oht
}
```

3.5 Live Scrolling Charts (Left & Right Sensor)

 [index.html](#) → Lines 388 – 452

Two Chart.js line charts are set up — one cyan (left sensor) and one red (right sensor). Both start pre-filled with 100 zeroes.

- animation: false — Disabling animation makes chart updates instant without any lag.
- pointRadius: 0 — Hides individual data dots so it looks like a smooth waveform.
- tension: 0.4 — Makes the line curve smoothly instead of being jagged/sharp.

```
const liveChartLeft = new
Chart(ctxLiveLeft, {
  type: 'line',
  data: {
    labels: Array.from({length: 100},
  (_, i) => i),
    datasets: [{
      data: Array(100).fill(0),
      borderColor: '#00f0ff',
      borderWidth: 2,
      pointRadius: 0,
      tension: 0.4
    }]
  },
  options: {
    animation: false,
    // ...
  }
});
```

3.6 Fleet Overview Table & Bar Chart

 [index.html](#) → Lines 700 – 771

The `updateHistoryUI()` function reads historical data and populates the table.

- Line 713: Calculates the minimum diameter across all 4 wheels (FL, FR, BL, BR) to find the worst wheel.
- Line 728: The threshold check — any min diameter at or below 123mm triggers a red WARNING status.
- Lines 750–758: Dynamically creates and injects a new table row (<tr>) for each OHT record.
- Lines 765–770: Updates the bar chart with the same data, coloring bars cyan (healthy) or red (warning).

```
function updateHistoryUI() {
  recentData.forEach(row => {
    const fl = parseFloat(row.FL_mm);
    const fr = parseFloat(row.FR_mm);
    const bl = parseFloat(row.BL_mm);
    const br = parseFloat(row.BR_mm);
    // Worst (smallest) wheel
    const minVal = Math.min(fl, fr, bl,
br);

    // Threshold check
    const isWarning = minVal <= 123.0;
    const statusText = isWarning
      ? 'WARNING (REPLACE)'
      : 'HEALTHY';

  });
}
```

3.7 OHT Analysis — Wear Trend Chart

 [index.html](#) → Lines 773 – 853

The `updateOhtTrendChart()` function filters data for a specific OHT and plots all 4 wheels over time.

- Line 816: Draws the flat dashed red threshold line at 123mm across the entire chart.
- Lines 818–828: Auto-scales the Y-axis to zoom in on the actual wear values, so the trend is always clearly visible.

```
// Draw threshold line at 123mm
ohtTrendChart.data.datasets[0].data
  = ohtData.map(() => 123.0);

// Auto-scale Y axis
const minVal = Math.min(...allNumericData,
123.0);
const maxVal = Math.max(...allNumericData,
123.0);
```

- Lines 848–852: Updates the 4 individual per-wheel charts (FL, FR, BL, BR) with the same filtered data.

```
ohtTrendChart.options.scales.y.min
  = Math.floor(minVal - padding);
ohtTrendChart.options.scales.y.max
  = Math.ceil(maxVal + padding);

ohtTrendChart.update();
```

3.8 The Live Polling Interval

 [index.html](#) → *Lines 651 – 670 (function) & Line 856 (trigger)*

This single line is what makes the entire dashboard feel "live".

`setInterval()` tells the browser to call `fetchLive()` every 100 milliseconds (10 times per second). Each time, it fetches the latest sensor readings from `/api/live` and updates the charts — all without ever refreshing the page.

Think of it like your browser sending a text to the Python backend 10 times a second asking "got any new numbers?" and the backend replying instantly.

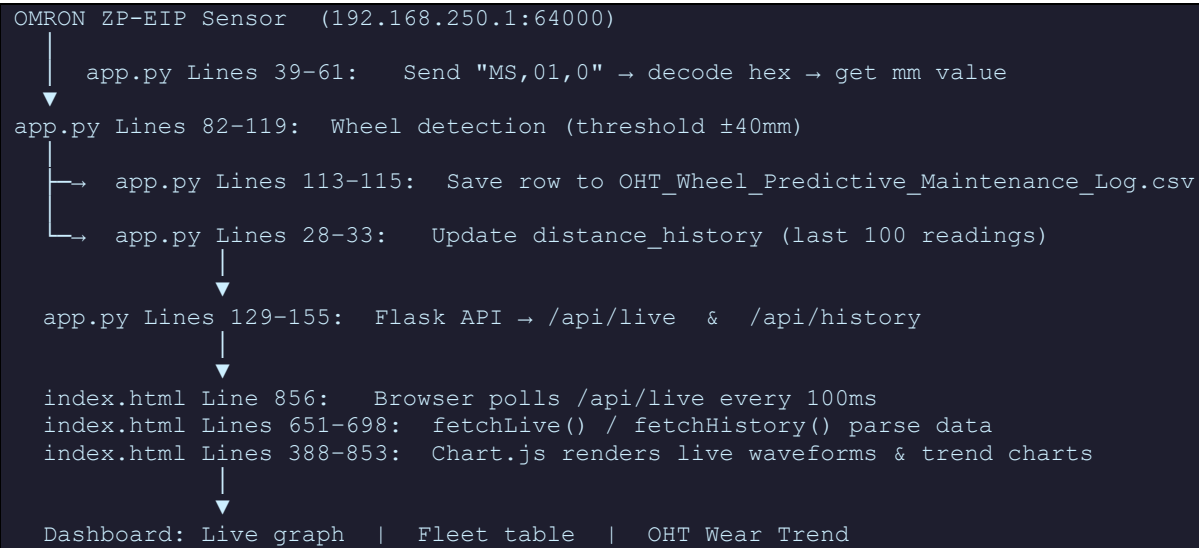
```
// Poll live data 10 times a second
setInterval(fetchLive, 100);

// fetchLive sends a request to
// the backend and updates charts:
async function fetchLive() {
  const res = await fetch('/api/live');
  const data = await res.json();

  liveChartLeft.data.datasets[0].data
    = data.left.history.map(
      d => d === 999.0 ? 0 : 125.0 +
      d
    );
  liveChartLeft.update();
}
```

4. Complete System Flow Summary

The diagram below shows how data travels from the physical sensor all the way to the browser dashboard:



Bottom line: This is a full end-to-end Predictive Maintenance monitoring system — from a physical laser sensor on a factory floor, all the way to a live browser dashboard — built entirely in Python and standard web technologies.